

Chat App Study Guide (Step by Step)

This guide explains your project in learning order, from setup to advanced realtime features. All examples are taken from your current codebase.

0) What You Built

You built a full realtime chat app using:

- React + Vite (frontend)
- Supabase Auth (signup/login/logout)
- Supabase Postgres (messages table)
- Supabase Realtime (live inserts/updates/deletes + presence)
- React Router (auth and chat routes)

Core abilities in your app:

- User signup and login
- Session-aware protected routes
- Create, read, update, delete messages
- Live message updates (no manual refresh)
- Online users (presence)
- Typing indicator
- Styled responsive UI
- Fallback polling when realtime is unstable

1) Project Setup (Phase 1)

1.1 Commands used

```
npm create vite@latest chat-app -- --template react
cd chat-app
npm install
npm install @supabase/supabase-js react-router-dom
```

1.2 Why these dependencies?

- `@supabase/supabase-js` : Client SDK to call Auth, DB, Realtime.
- `react-router-dom` : Browser routing between auth and chat pages.

1.3 Package scripts

From `package.json` :

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview"
}
```

- `npm run dev` starts development server.
 - `npm run build` checks production build.
-

2) App Entry and Routing Base

File: `src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

What this does

1. React mounts into `<div id="root">` .
 2. `BrowserRouter` enables route navigation (`/auth` , `/chat` , etc.).
 3. Global styles are imported (`index.css`).
-

3) Supabase Client Setup

File: `src/supabaseClient.js`

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error(
    'Missing Supabase environment variables. Add VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY to .env',
  )
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

Key learning

- Vite env variables must start with `VITE_` .
- If keys are missing, app throws early to avoid silent broken UI.

- `createClient(url, anonKey)` gives one SDK instance used everywhere.
-

4) Database and Security (Phase 3)

File: `supabase/setup.sql`

4.1 Table schema

```
create table if not exists public.messages (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  content text not null check (char_length(trim(content)) > 0),  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

- `id` : unique message ID.
- `user_id` : owner, linked to authenticated Supabase users.
- `content` : non-empty message body.
- `created_at` : message time.
- `updated_at` : updated by trigger on edits.

4.2 Trigger for update timestamp

```
create or replace function public.set_updated_at()  
returns trigger  
language plpgsql  
as $$  
begin  
  new.updated_at = now();  
  return new;  
end;  
$$;  
  
create trigger messages_set_updated_at  
before update on public.messages  
for each row  
execute function public.set_updated_at();
```

This auto-updates `updated_at` whenever a row is edited.

4.3 RLS policies

```
alter table public.messages enable row level security;
```

Then policies:

- Read: all authenticated users can select.
- Insert: user can insert only with `auth.uid() = user_id`.

- Update/Delete: only owner can update/delete own messages.

This is critical for security.

4.4 Realtime publication

```
alter publication supabase_realtime add table public.messages;
```

Without this, DB changes will not emit realtime events.

5) Auth Flow (Phase 2)

Auth code is in `AuthPage` component inside `src/App.jsx`.

5.1 Signup/Login API usage

```
if (mode === 'signup') {
  result = await supabase.auth.signUp(credentials)
} else {
  result = await supabase.auth.signInWithPassword(credentials)
}
```

5.2 What happens on submit?

1. Prevent default form submit.
2. Reset notices/errors.
3. Call Supabase auth method.
4. If error: show message.
5. If signup requires email verification: show notice.
6. Else navigate to `/chat`.

5.3 Session bootstrapping in root `App`

```
const { data, error } = await supabase.auth.getUser()
setUser(data.user ?? null)
```

And live auth listener:

```
supabase.auth.onAuthStateChange((_event, session) => {
  setUser(session?.user ?? null)
})
```

This keeps UI synced when user logs in/out.

6) Route Protection

In `App` return block:

- `/` redirects to `/chat` if logged in, otherwise `/auth`.

- `/auth` redirects to `/chat` when already logged in.
- `/chat` redirects to `/auth` when not logged in.

This pattern is your route guard.

7) Chat State Design (Phase 4/5)

Inside `ChatPage` you manage:

- `messages` : all fetched messages
- `draft` : current input text
- `loadingMessages` : first load spinner
- `sending` : disable send button while sending
- `onlineUsers` : presence list
- `typingUsers` : typing indicator source
- `editingId` + `editingText` : edit mode state

Why this matters:

- Each state is focused and single-purpose.
 - UI behavior becomes predictable and testable.
-

8) Fetch Messages (Phase 5)

```
const { data, error } = await supabase
  .from(MESSAGES_TABLE)
  .select('*')
  .order('created_at', { ascending: true })
```

Pattern used

- First call: `fetchMessages({ showLoader: true })`
- Polling fallback: `setInterval(() => fetchMessages({ silent: true }), 2500)`

This means:

- Initial load shows loader.
 - Background sync updates silently.
 - Even if realtime has delays, UI still updates.
-

9) Send Message (Phase 4)

```
const { data, error } = await supabase
  .from(MESSAGES_TABLE)
  .insert([
    {
      user_id: user.id,
      content,
    },
  ])
```

```
.select()
.single()
```

Important details:

- `trim()` avoids blank messages.
 - `sending` flag prevents double-submit race.
 - After success, draft clears and typing false is tracked.
-

10) Realtime Core (Phase 6)

Realtime subscription setup:

```
const channel = supabase
  .channel(ROOM_CHANNEL, {
    config: { presence: { key: user.id } },
  })
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table: MESSAGES_TABLE }, ...)
  .on('postgres_changes', { event: 'UPDATE', schema: 'public', table: MESSAGES_TABLE }, ...)
  .on('postgres_changes', { event: 'DELETE', schema: 'public', table: MESSAGES_TABLE }, ...)
  .on('presence', { event: 'sync' }, ...)
  .on('presence', { event: 'join' }, ...)
  .on('presence', { event: 'leave' }, ...)
```

Why this is powerful

- INSERT event: add new row instantly.
 - UPDATE event: edit reflects instantly.
 - DELETE event: remove instantly.
 - Presence events: online list and typing update.
-

11) Online Users and Presence (Phase 8)

You parse presence in a robust way:

```
const rawState = channel.presenceState()
const flattenedState = Object.entries(rawState)
  .flatMap(([presenceKey, entryValue]) => {
    const metas = Array.isArray(entryValue)
      ? entryValue
      : Array.isArray(entryValue?.metas)
        ? entryValue?.metas
        : entryValue && typeof entryValue === 'object'
          ? Object.values(entryValue)
          : []

    if (!metas.length) {
      return [{ userId: presenceKey, email: undefined, typing: false }]
    }
  })
```

```

return metas.map((meta) => ({
  userId: meta.user_id ?? presenceKey,
  email: meta.email,
  typing: Boolean(meta.typing),
}))
})

```

Why this exists:

- Supabase presence payload shape can vary.
- Fallbacks prevent always-0 online count.

Track self presence with retry:

```

let status = await channel.track({ user_id: user.id, email: user.email, typing: false })
if (status !== 'ok') {
  await new Promise((resolve) => setTimeout(resolve, 300))
  status = await channel.track({ user_id: user.id, email: user.email, typing: false })
}

```

12) Typing Indicator (Phase 8)

handleDraftChange behavior:

1. Update draft state.
2. Clear old timeout.
3. If empty input -> typing: false .
4. Else track typing: true .
5. After 1 second idle -> track typing: false .

Label render:

```

const typingLabel =
  typingUsers.length > 0
    ? `${typingUsers.map((entry) => toHandle(entry.email, entry.userId)).join(', ')} ${
      typingUsers.length > 1 ? 'are' : 'is'
    } typing...`
    : ''

```

13) Edit and Delete Messages (Phase 8)

Delete

```

await supabase
  .from(MESSAGES_TABLE)
  .delete()
  .eq('id', id)
  .eq('user_id', user.id)

```

- Optimistic UI: remove from local list first.
- On error: rollback using previous state.

Edit

```
await supabase
  .from(MESSAGES_TABLE)
  .update({ content })
  .eq('id', id)
  .eq('user_id', user.id)
  .select()
  .single()
```

- Owner-only enforced by both query and RLS.
- Local message list replaced with returned updated row.

14) Scroll and UX details (Phase 7/9)

```
bottomAnchorRef.current?.scrollIntoView({ behavior: 'smooth', block: 'end' })
```

Runs when `messages` change, so latest message stays visible.

Other polish already present:

- Loader during session check and first message fetch
- Empty state text
- Error banner for auth/realtime failures
- Disabled states while sending/loading

15) Styling System

You use:

- CSS variables in `src/index.css`
- Component styles in `src/App.css`
- Responsive media queries
- Gradient and blur visual layers

This gives consistent design tokens and easier theme edits.

16) Cleanup and Memory Safety

In effect cleanup:

```
clearTimeout(typingTimerRef.current)
clearInterval(pollTimerRef.current)
if (channelRef.current) {
  supabase.removeChannel(channelRef.current)
```



```
channelRef.current = null  
}
```

Why this is important:

- Prevent memory leaks
- Prevent duplicate subscriptions
- Prevent stale timers after unmount

17) Common Issues and Fixes

Problem: Messages only update after reload

Reason: Realtime not firing. Fixes in your code:

- Realtime INSERT/UPDATE/DELETE handlers
- Poll fallback every 2.5s

Problem: Online users stays 0

Possible reasons:

- Presence payload shape mismatch
- Presence track failure
- Realtime/presence disabled in Supabase

Fixes in your code:

- Resilient payload parsing
- Track retry
- Fallback self insertion

18) How to Run Correctly

Always run from project folder:

```
cd D:\coding\webdev\React\chat-app  
npm run dev
```

If port is busy, Vite auto-picks next port.

19) Suggested Learning Order

1. Read `src/supabaseClient.js`
 2. Read auth parts in `src/App.jsx`
 3. Read route guards in `src/App.jsx`
 4. Read SQL in `supabase/setup.sql`
 5. Read `fetchMessages` and send/edit/delete logic
 6. Read realtime and presence logic
 7. Read CSS tokens and responsive styles
-

20) Mini Exercises for Practice

1. Add room support (`rooms` table + `room_id` in messages).
 2. Add message seen status.
 3. Add profile table with username/avatar.
 4. Replace polling fallback with reconnect strategy only.
 5. Add unit tests for helper functions (`toHandle` , message reducers).
-

21) Final Architecture Summary

High-level flow:

1. User opens app.
2. App checks session.
3. If logged out -> Auth page.
4. If logged in -> Chat page.
5. Chat loads messages + starts realtime channel + tracks presence.
6. User sends/edits/deletes messages.
7. UI updates from realtime events; fallback poll guarantees freshness.

You now have a production-style foundation for realtime apps.